

ABYSS

게임 콘텐츠를 구현하고,
변경에 강한 시스템을 설계합니다.

Unity · C# 게임 프로그래밍 포트폴리오

UNITY 6

C#

PURE CORE

SAVE v27

GAME PROGRAMMING



ABYSS / 전투 화면

CODE / SYSTEMS / RELIABILITY

GAME PROGRAMMER / PROJECT PORTFOLIO

ABYSS 프로젝트 소개

다키스트 던전 스타일의 2D 턴제 RPG. 전투·탐험·영지 운영을 하나의 캠페인으로 구현했습니다.



원정 지도 / 던전 선택과 1~4인 파티 편성

프로젝트 구현 범위

C# 전투·탐험·캠페인 시스템, 저장·복원, Unity UI 연결, 개발 도구 및 검증 자동화

사용 기술

C# · Unity 6 · URP · uGUI · Input System
JSON · .NET · NUnit · Git · PowerShell · Codex

게임 흐름

편성·보급 → 탐험·전투 → 성공 귀환 / 후퇴
→ 결산·성장·영지 운영

콘텐츠·시스템 구현

전투 효과 실행기와 설정 기반 코어

코드 분석·문제 해결

의존성 분리, 난수 격리, UI 버그 수정

도구·자동화

썬 생성, 데이터 검증, AI 결과 검증

규칙과 화면의 책임을 분리한 구조

객체지향 설계와 의존성 분리를 통해 게임 로직을 Unity와 독립적으로 실행·검증합니다.

Unity Presentation

화면·입력·씬 전환



Application

CampaignSession / 화면 서비스 / 불변 DTO



Content

JSON 로드·검증 / 게임별 설정·전투 투영



Infrastructure

저장 codec / 파일 IO / JSON 지원



Rules

전투·탐험 계산 / 도메인 모델 / 이벤트

왜 분리했는가

씬·프레임·UI 이벤트와 전투 계산이 섞이면 재현과 회귀 확인이 어려워집니다. 같은 규칙을 콘솔과 Unity에서 실행하도록 경계를 만들었습니다.

경계를 강제하는 장치

Core asmdef의 noEngineReferences와 A15 참조 게이트로 의존성을 검사합니다. Rules에서 파일 IO·벽시계·직접 RNG 생성도 금지합니다.

직접 참조 집합

Rules	없음
Infrastructure	Rules
Content	Rules, Infrastructure
Application	Rules, Infrastructure, Content
Presentation	Rules, Content, Application

화살표: 핵심 계층 방향. 정확한 직접 참조는 우측 표 기준.

설계 결과 / 전투 계산은 헤드리스 환경에서 검증하고, 화면은 조회 DTO와 명령 서비스로 연결합니다.

SOURCE / Core/Rules/Abyss.Core.Rules.asmdef · verify.ps1

11종 스킬 효과를 하나의 실행 경로로

효과 객체와 실행 단계를 분리해 버프·이동·치료·소환을 조합할 수 있도록 구현했습니다.



Data/skills.json:221 / 중복 effects 발체

```

"effects": [
  { "type": "buff",
    "phase": "after_primary_resolution",
    "buffId": "PROT10", "turns": 2 }
]
  
```

축복은 기본 치유 판정 뒤 대상에게 2턴 방어 버프를 적용합니다. 기본 명중·피해·치유는 전용 판정 경로에 남겨둡니다.

확장 방식

Buff, Push/Pull, Cure, Summon을 기존 기본 판정과 같은 경로에 연결하고, C#의 옛 효과 생성자·변환 표면을 제거했습니다.

실행 순서의 일관성

BattleEngine이 단계와 목록 순서를 관리하고, 실제 효과는 effect.Apply(context)에 위임합니다. 효과를 조합해도 사건과 난수 소비 순서를 유지합니다.

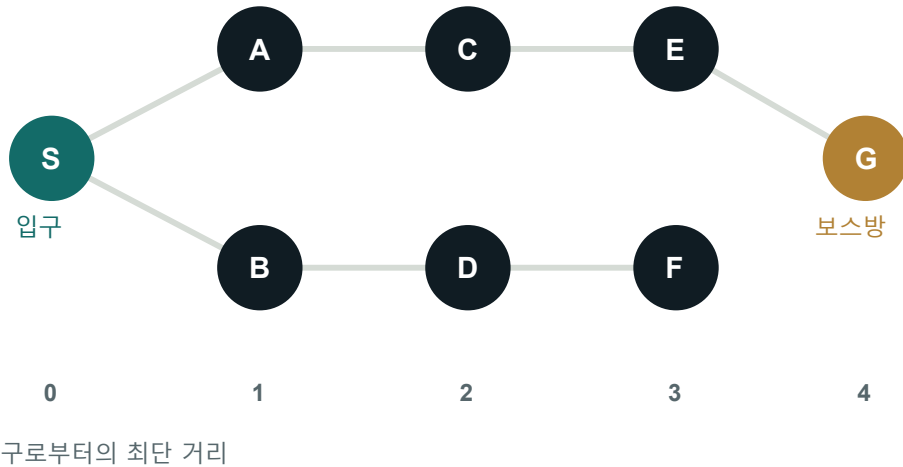
검증

빛나감·반격 사망의 사건 순서, 이동 불가 시 난수 0회, 다중 대상에서도 시전자 이동·소환 1회 실행을 검사합니다.

그래프 탐색으로 던전의 연결성을 보장

방과 복도를 그래프로 모델링하고, BFS로 도달 가능성과 입구에서의 최단 거리를 계산했습니다.

던전 그래프 / 탐색 구조 예시



연결되지 않은 방 보정

Flood 탐색으로 허브에서 도달 가능한 방을 찾고, 인접 칸 연결과 고립 방 정리로 모든 방의 연결성을 보장합니다.

BFS 기반 보스방 배치

Queue 기반 탐색으로 최단 거리를 구한 뒤 입구에서 가장 먼 방을 보스방으로 선택합니다. 동률은 안정 ID 순서로 결정합니다.

자료구조 선택

Queue와 거리 Dictionary로 중복 방문을 막습니다. BFS의 시간 복잡도는 인접 관계 탐색 기준 $O(V + E)$ 입니다.

알고리즘 검증

40개 시드에서 모든 방 도달성 검사 / 20개 시드에서 보스방 최장 거리 검사
같은 시드의 생성 결과를 반복 비교해 지도 생성의 재현성을 확인했습니다.

재현 가능한 전투와 난수 격리

버그를 같은 입력으로 다시 실행하고, 기능 변경이 다른 시스템에 미치는 영향을 통제합니다.

문제

UI 조회나 다른 시스템의 난수 호출이 전투 결과를 바꾸면, 버그를 재현하거나 저장 전후 결과를 비교하기 어렵습니다.

설계

IRandomSource를 주입하고 PCG32의 상태를 저장합니다. 전투·탐험·전리품 등 7개 named stream을 분리해 소비 경로를 명시합니다.

campaign

state + increment

town

state + increment

dungeon

state + increment

encounter

state + increment

combat

state + increment

ai

state + increment

loot

state + increment

RandomStreamTests.cs:47-48 / 실제 코드

```
RandomStreams changed = new RandomStreams(42UL);
Draw(changed.Get(RandomStreams.Names.Loot), 500);
```

전리품 난수 500회 추가 소비 후에도
전투·던전 난수 수열이 동일한지 대조합니다.

저장 재개 검증

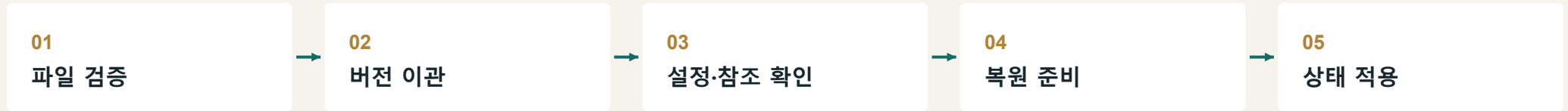
1·2·3·7·11·19턴, 여섯 지점에서 전투를 중단·복원한 결과를 연속 실행과 대조합니다. 모든 스트림 상태의 복원도 검사합니다.

설계 효과와 경계

전리품 변경이 전투 난수에 주는 영향을 격리합니다. 같은 스트림 내부의 소비 순서는 규칙 계약으로 유지합니다.

실패 원자성과 저장 호환성

저장·복원 과정이 중단되더라도 현재 상태를 보존하도록 검증과 적용 순서를 설계했습니다.



호환성은 버전 번호만으로 판단하지 않습니다

v27 저장은 콘텐츠 revision과 실행 설정 서명을 사용합니다. 진행 전투의 규칙·진형 크기와 원정 설정이 다르면 적용 전에 거절합니다.

과거 규칙은 고정값으로 보존합니다

마이그레이션은 현재 프리셋 대신 당시의 동결 서명을 사용합니다. 재개 의미가 달라지는 설정은 당시 서명과 비교해 호환 여부를 판정합니다.

슬롯 파일 쓰기

임시 파일 작성 → **Flush(true)** →
대상 파일 **Replace / Move** → 백업 정리

Core/Infrastructure/SaveStore.cs:56

실패 원자성 검증

활동·병동·예약 손상 26종 × 2개 RNG 복원 경로, 총 52개 시나리오에서 영지·RNG·기존 영웅 참조가 바뀌지 않는지 확인했습니다.

영웅 설정까지 연결

기술 선택·장신구 슬롯·보관함을 보존합니다. 저장 실패 시 영웅 설정 명령의 변경도 되돌립니다.

설계 결과 / 잘못된 저장 데이터는 적용 전에 차단하고, 파일 기록과 메모리 상태의 일관성을 유지합니다.

게임 상태와 사용자 입력을 연결하는 UI

영웅 정보 조회, 기술 선택, 장신구 교체를 Application 서비스와 연결했습니다.



영웅 상태창 / 능력치·기술·장비·기백 정보

씬 제작: production builder로 씬·프리팸을 생성하고 기존 .meta / GUID를 유지합니다.

아트 연결: 안정적인 직업 ID로 스프라이트를 선택하고, 누락 시 문자 표시로 조작 기능을 유지합니다.

01 조회가 게임을 바꾸지 않도록

상태창 반복 조회가 캠페인·7개 RNG·저장 파일을 바꾸지 않는지 검사합니다.

02 설정 명령을 실제 저장에 연결

전투 기술 1~4개 제한과 장신구 장착·교체·해제를 서비스로 요청하고 실패 시 복구합니다.

03 입력과 해상도도 계약으로

Esc 닫기 순서, 보관함 항목 제거 후 초점 복구, 1080p/720p 고정 영역·글자 잘림을 확인합니다.

사망한 몬스터가 HP 1로 보이던 문제 해결

표시 오류를 추적해 생존 체력과 시체 내구도의 의미를 분리했습니다.



수정 결과 / 사망 HP 0과 시체 내구도를 분리 표시

BEFORE

HP 1 / 28

AFTER

HP 0 / 28

시체 내구도 1 / 1

증상

사망한 적이 HP 1/28로 표시되어 살아 있는 것처럼 보였습니다.

원인

시체 전환 후 CurrentHealth는 파괴용 내구도 1인데, UI가 생존 최대 HP와 함께 표시했습니다.

수정

Application 조회에 LivingHealth / CorpseHealth / CorpseMaxHealth를 분리하고 표시 의미를 명확히 했습니다.

해결 결과

사망 HP는 0으로, 파괴 가능한 시체는 내구도로 구분했습니다. 전투 규칙·난수·저장 형식을 유지하며 조회 모델과 표시를 수정했습니다.

반복 작업을 줄이는 개발 도구

콘텐츠 편집·씬 구성·코드 검증을 도구로 묶어, 변경 결과를 반복해서 확인할 수 있게 만들었습니다.

01 데이터 검증기

JSON 허용 필드·타입·ID 참조를 검사하고, 오류 위치를 파일과 전체 경로로 알려줍니다.

02 씬·프리팹 생성기

에디터 메뉴에서 Start·Loading·Estate·Dungeon을 생성하고, 기존 GUID와 참조를 유지합니다.

03 통합 검증 스크립트

PowerShell로 빌드·규칙 테스트·플레이 흐름·어셈블리 참조 검사를 한 번에 실행합니다.

콘텐츠 수정과 로직 수정을 분리

영웅·스킬·몬스터·시설·이벤트를 JSON으로 정의했습니다. 지원 스키마 안의 콘텐츠 추가와 밸런스 조정은 C# 수정 없이 진행할 수 있습니다.

씬 구성의 재현성 확보

Builder가 UI 계층과 프리팹 배선을 생성합니다. 저장소의 Data를 StreamingAssets에 동기화하여 콘텐츠 원본과 실행용 파일을 구분합니다.

콘텐츠 진단 예시 / 영웅이 존재하지 않는 기술을 참조한 경우

```
heroes.json:$.heroes[0].skills[0]
```

로딩 단계에서 원인과 위치를 식별해, 잘못된 데이터가 전투 실행 오류로 이어지는 것을 차단합니다.

생성형 AI를 활용한 구현과 검증

Codex를 코드 작성·리팩터링·테스트 작성에 활용하고, 규칙 명세와 실행 결과를 기준으로 검증했습니다.

83,449

Core 회귀 assertion 통과

규칙·저장·콘텐츠·Application

0

구조·완료 조건 위반

의존성 및 플레이 흐름 검사

0 / 0

Core 빌드 경고 / 오류

2026.09.07 검증 결과

01 요구사항 명세화

규칙 결정표와 계층별 책임을 코드 작성 기준으로 사용

02 AI 보조 구현

기능을 나누어 코드·리팩터링·회귀 테스트 작성에 활용

03 변경 결과 검증

빌드·규칙 테스트·구조 게이트와 기존 기준선을 대조

리팩터링 사례: 기존 규칙 설정으로 원정·6주·12주 캠페인을 실행해 동결 기준선과 비교했습니다. 사건 순서·난수 상태·최종 저장을 대조해 설정 외부화 전후의 동작이 유지되는지 확인했습니다.

주요 코드 및 설계 사례

독립 실행 코드, 테스트와 핵심 구현 발채를 제출용 저장소에 정리했습니다.

GITHUB REPOSITORY

github.com/darkeye3/Abyss-Portfolio

전투 시스템	효과 실행 순서 · 코드 발채와 설계	VIEW
자료구조·알고리즘	BFS 거리 계산 · 독립 실행 샘플	VIEW
결정론	7개 난수 스트림 · 원본 구현 발채	VIEW
저장 안정성	사전 검증·원자적 교체 · 코드 발채	VIEW
Unity·개발 도구	UI 문제 해결·씬 생성·콘텐츠 검증	VIEW
테스트	공개 샘플 실행 방법과 테스트 구성	VIEW

게임 규칙을 구조화하고, 재현 가능한 코드와 도구로 구현합니다.